On-prem setup runbook

APIConnect-catalog on Azure DevOps Server with a self-hosted Linux agent

v1.5 · 12 October 2026
companion to guide.pdf (sections 12–13)

This runbook takes you from the offline bundle to the first successful publish on an **on-prem Azure DevOps Server** (2020 or later; 2022 recommended) using **Azure Repos Git** and a **self-hosted Linux x64 agent** that has no internet access. Every step lists *where* to click and *what* to enter. Estimated time: 2–3 hours, most of it waiting for API keys and permissions.

What is different on Azure DevOps Server

Topic	On Azure DevOps Server
PR validation	The `pr:` block in `apic-ci.yml` is ignored for Azure Repos. PR builds are configured with a **branch policy** → **Build validation** on `main` (step 9).
Secrets	No Azure Key Vault. `APIC_APIKEY` is a **secret variable** (padlock) in the variable group; it is encrypted at rest and masked in logs.
Agents	No Microsoft-hosted pool. Set `AGENT_POOL` in `apic-ci.yml` to your self-hosted pool (step 6). Tools come from the bundle (`TOOLS_MODE=offline`).
Toolkit	Installed on the agent; `APIC_TOOLKIT_SOURCE=preinstalled`. The Azure Artifacts task is not even included in the compiled pipeline.
Environments / deployment jobs	Available since Server 2020. The five `apic-*` Environments are created automatically on the first run.
Certificates	Internal CAs are common on-prem: the agent must trust the Azure DevOps Server certificate *and* the three API Connect management ingresses (step 3).

You will need

- The bundle `APIConnect-catalog-bundle-<date>.zip` and its `.sha256`.
- A Linux x64 VM for the agent (2 vCPU / 4 GB is plenty): bash 5, git ≥ 2.30, Python 3.9+ with pip, `unzip`, `sha256sum`, outbound HTTPS to the Azure DevOps Server and to the three API Connect management ingresses.
- From each API Connect instance's API Manager: the Linux CLI, `credentials.json` and a long-lived toolkit API key (3×) — step 7 explains how to obtain them.
- A technical user per instance that is a member of the provider org with a publish role, and a Cloud Manager admin available once per instance (to allow reusable API keys).
- Azure DevOps Server: rights to create a project/repo, agent pool, variable groups, secure files, pipelines and branch policies (Project Administrator).

Part A — The agent machine

1 Install the bundled tools

Where: the agent VM, as a user with sudo

```
sha256sum -c APIConnect-catalog-bundle-*.zip.sha256          # integrity of the transfer
unzip APIConnect-catalog-bundle-*.zip && cd APIConnect-catalog-bundle-*
sudo bash tools/install-offline.sh                           # verifies SHA256SUMS, installs
yq/oasdiff/jq/spectral to /usr/local/bin, pip installs yamllint
```

On Debian 12+ / Ubuntu 23.04+ the system Python is "externally managed" (PEP 668); the script then retries with `--break-system-packages` (intended for a dedicated agent image) and, as a last resort, `--user` — in that case `~/.local/bin` of the *agent* user must be on PATH (see step 4).

2 Install the IBM API Connect toolkit

Where: API Manager of any instance (use the highest fix pack) → *Tools for download* → CLI (Linux); then the agent VM

```
sudo install -m 0755 apic-slim /usr/local/bin/apic           # or `apic` if you downloaded CLI
+ Designer
apic --accept-license --live-help=false version              # must print the toolkit version,
10.0.8.x - run it as the AGENT user too
```

> **Two first-run questions — answer both by flag, or every call blocks**
>
> On a HOME without `~/.apiconnect` the toolkit asks two things on stdin: the license (`--accept-license`) and "live help" (`--live-help=false`). A pending question in a pipeline step — where stdin is a pipe nobody writes to — looks like a silent hang whose output appears only after *Cancel* closes the pipe. `--accept-license` alone is not enough: the second question stays pending, nothing is persisted and the hang returns on the next call. With **both** flags the call completes and the answers are written to `~/.apiconnect` *of the user that ran it*; after that, plain `apic` works for that user. Hence: put `RUN apic --accept-license --live-help=false version` in the agent Dockerfile (same user the agent runs as — a `USER` switch after it, or a different `HOME`, brings the prompt back), and keep the repository scripts, which pass both flags on every call anyway (`scripts/lib.sh`: flags, stdin from `/dev/null`, timeout `APIC_TIMEOUT`, default 600 s). If you call `apic` yourself in a pipeline step, write `apic --accept-license --live-help=false …`.

Prove the agent image before the first pipeline run (Docker on any machine that has the image and the repository): `docker run --rm -v "$PWD:/src:ro" <agent-image> bash /src/tests/e2e/run-tests.sh` — runs the change-set logic and stages 1–4 against the real toolkit with stdin held open exactly like an agent step, shows how the first-run questions behave on this image, and skips everything that would write to API Connect.

3 Trust the internal certificates

Where: the agent VM. Skip if all endpoints use publicly trusted certificates.

```
# OS trust store (git, curl, most tools). Ubuntu/Debian:
sudo cp corp-root-ca.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates
# RHEL:  sudo cp corp-root-ca.crt /etc/pki/ca-trust/source/anchors/ && sudo update-ca-trust
# Node-based tools (the Azure Pipelines agent tasks, and the apic toolkit - VERIFY for apic):
echo 'NODE_EXTRA_CA_CERTS=/etc/ssl/certs/ca-certificates.crt' | sudo tee -a /etc/environment
```

Test from the VM before going further: `curl -sS https://<ado-server>/tfs/ -o /dev/null -w '%{http_code}\n'` and `apic catalogs:list --server <ingress.management> --org <org>` after an interactive `apic iam-apikey` login. If `apic` reports a TLS error while `curl` works, the toolkit does not honour the OS store on your build; ask IBM support for the supported CA option of your toolkit version.

4 Register the Azure Pipelines agent

Where: Azure DevOps Server → *Collection settings* → *Agent pools* → *Add pool* (self-hosted, name `apic-linux`) → *New agent* → download the Linux x64 agent (it is served by your own server). Create a PAT with scope *Agent Pools (Read & manage)* for the registration.

```
sudo useradd -m -s /bin/bash azagent                          # dedicated user; never root
sudo mkdir -p /opt/azagent && sudo chown azagent /opt/azagent
sudo -iu azagent bash -c 'cd /opt/azagent && tar zxf ~/vsts-agent-linux-x64-*.tar.gz && \
  ./config.sh --unattended --url https://<ado-server>/tfs/DefaultCollection --auth pat --token
<PAT> \
              --pool apic-linux --agent apic-agent-01 --acceptTeeEula --replace'
# PATH for the service (systemd does not read the user's profile):
echo 'PATH=/usr/local/bin:/usr/bin:/bin:/home/azagent/.local/bin' | sudo -u azagent tee
/opt/azagent/.env
cd /opt/azagent && sudo ./svc.sh install azagent && sudo ./svc.sh start
# verify as the agent user: every line must print a path
sudo -iu azagent bash -c 'for t in yamllint spectral yq oasdiff jq apic git realpath; do
command -v $t || echo "MISSING $t"; done'
```

The pool must have *Grant access permission to all pipelines* (pool → Security), or you authorize the pipeline on its first run.

Part B — Repository and configuration

5 Import the repository into Azure Repos

Where: Azure DevOps Server → your project → *Repos* → *New repository* (Git, name `APIConnect-catalog`, **uncheck** "Add a README"); then any machine with git

```
git clone APIConnect-catalog.gitbundle APIConnect-catalog     # from the bundle, with history
cd APIConnect-catalog
git remote add origin https://<ado-server>/tfs/DefaultCollection/<project>/_git/APIConnect-catalog
git push -u origin main
```

Then set `main` as the default branch (*Repos* → *Branches* → … → *Set as default branch*) if it is not already.

Use the `.gitbundle`, not the plain `repo/` folder, when the pushing machine is Windows: `git add` on Windows stores every file as `644`, so the scripts lose their exec bit. The pipeline invokes every script through `bash` and does not depend on that bit, but if you did import `repo/`, restore it anyway: `git update-index --chmod=+x scripts/*.sh tools/*.sh && git commit -m "exec bits"`.

6 Point the repository at your instances and your pool

Where: a branch + PR, or directly on `main` before the branch policy exists

File · setting	Value
`config/topology.yml` · `instances.dev.ingress.management`	The `apic --server` URL of the dev instance (platform-API / API Manager ingress), e.g. `https://api-manager.dev.apic.corp.local`. Same for `test` and `prod`.
`config/topology.yml` · `instances.*.ingress.gateway`	The gateway ingress of each instance (informational, used in docs/smoke tests).
`config/topology.yml` · `instances.*.org`	Provider organization name on each instance (`apic orgs:list --server …` shows it).
`.azure-pipelines/apic-ci.yml` · `AGENT_POOL`	`apic-linux` (the pool from step 4)
`.azure-pipelines/apic-ci.yml` · `APIC_TOOLKIT_SOURCE`	`preinstalled` (already the default)

No `CHANGE ME` may remain in `topology.yml`: the pipeline warns on placeholders. Commit and push.

7 Get the API Connect credentials — per instance (dev, test, prod)

Where: API Manager of each instance (browser) and a shell with the toolkit, e.g. the agent VM. Two things are needed per instance: `credentials.json` (identifies the *toolkit* as an OAuth client) and `APIC_APIKEY` (identifies the *user* the pipeline publishes as).

7a · credentials.json

1. Open `https://<management-ingress>/manager`, log in.
2. Home page → **Download toolkit** (older builds: *Tools for download / Help → Download toolkit*).
3. In the dialog, *Step 2: Download credentials* → **Download** next to *Credential*. Ignore *API Designer Credentials*.
4. Rename the file per instance — the name is how the pipeline finds it: `apic-toolkit-credentials-dev.json`, `apic-toolkit-credentials-test.json`, `apic-toolkit-credentials-prod.json`.

The file is per instance (dev's does not work against test's manager) and contains no user identity. If the download link is missing, your user lacks the permission — a Cloud Manager admin can download it from *Cloud Manager → Settings → Toolkit*.

7b · APIC_APIKEY — a long-lived toolkit API key

The key is issued by the instance itself. Keys from the browser sign-in page (`…/auth/manager/sign-in/?from=TOOLKIT`) are single-use and expire in minutes — fine for a manual test, useless for a pipeline. Create a long-lived one instead (commands verified against the v10.0.8 CLI reference):

```
MGMT=https://<management-ingress>      # ingress.management of this instance in
config/topology.yml
ORG=<provider-org>

# 1. Allow reusable keys - Cloud Manager admin, once per instance
cat > cloud-settings.yaml <<'EOF'
api_key_enabled: true
api_key_multiple_uses: true
api_key_expires_in: 31536000
EOF
apic login --server "$MGMT" --sso --context admin          # open the printed URL, log in,
paste the key back
apic cloud-settings:update --server "$MGMT" cloud-settings.yaml

# 2. Create the key - logged in as the technical user the pipeline will publish as
cat > pipeline-key.yaml <<'EOF'
client_type: toolkit
name: azure-pipeline
description: Azure DevOps publish key
ttl: 31536000
EOF
apic login --server "$MGMT" --sso --context provider       # browser URL again, as the
technical user
apic api-keys:create --server "$MGMT" pipeline-key.yaml    # output contains  api_key:
xxxxxxxx-....  = APIC_APIKEY

# 3. Test key + credentials.json together
apic logout --server "$MGMT"
apic client-creds:clear && apic client-creds:set ./apic-toolkit-credentials-dev.json
apic login --server "$MGMT" --sso --context provider --apiKey '<api_key>'
apic catalogs:list --server "$MGMT" --org "$ORG"            # must list the project catalogs
```

- `ttl: 31536000` = 1 year. Put the expiry date in your calendar; an expired key fails the login step with an unhelpful message.
- The key inherits the permissions of the user who created it: that user must be a member of the provider org with a publish-capable role (Developer / Administrator, or a custom role with `product:manage`).
- Registry without SSO (local user registry or LDAP)? Then there is no API key: the pipeline needs `--username / --password / --realm provider/<idp>` instead — `apic identity-providers:list --server "$MGMT" --scope provider --fields realm` shows what you have. Adjust the login step accordingly.

> **Match the login command to the key**
> The pipeline's login step in `templates/publish-stage.yml` ships as `apic iam-apikey …`, which is for IBM Cloud IAM keys (SaaS). For a toolkit key created as above the line must be `apic login --server "$APIC_SERVER" --sso --context provider --apiKey "$APIC_APIKEY"` — change it in a PR before the first publish.

8 Library: variable groups and secure files

Where: *Pipelines → Library*

Object	Content
Variable group `apic-shared`	`TOOLS_MODE` = `offline` · `APIC_TOOLKIT_VERSION` = e.g. `10.0.8.9` (informational)
Variable group `apic-dev`	`APIC_APIKEY` = the dev instance's key from step 7b — **click the padlock** to make it secret
Variable group `apic-test`	`APIC_APIKEY` for the test instance (secret)
Variable group `apic-prod`	`APIC_APIKEY` for the prod instance (secret)
Secure files (tab *Secure files*)	**+ Secure file** → upload the three files from step 7a: `apic-toolkit-credentials-dev.json`, `apic-toolkit-credentials-test.json`, `apic-toolkit-credentials-prod.json`.

On each group and file: *Pipeline permissions* → either authorize the `APIConnect-catalog` pipeline after you created it (step 9), or "Open access" if your policy allows. Restrict who can *edit* `apic-prod` (group → Security).

Part C — Pipeline, policies, permissions

9 Create the pipeline and the PR policy

Where: *Pipelines → New pipeline → Azure Repos Git → APIConnect-catalog → Existing Azure Pipelines YAML file* → path `/.azure-pipelines/apic-ci.yml` → **Save** (not Run). Rename it to `APIConnect-catalog` (… → Rename/move).

Then *Repos → Branches → main → … → Branch policies*:

- **Require a minimum number of reviewers**: 1 (or more), "Allow requestors to approve their own changes" off.
- **Build validation** → **+**: pipeline `APIConnect-catalog`, trigger *Automatic*, policy requirement *Required*, build expiration *Immediately when main is updated*. This is what runs the four validation stages on every PR (the YAML `pr:` block is ignored on Azure Repos).
- Optional: **Automatically included reviewers** for `projects/*/prod/**` (the platform team) — a lightweight prod gate without pipeline approvals.

> **Effect of the policy**
> Nobody can push to `main` directly anymore — including you. Every change, including `topology.yml` edits, goes through a PR. That is intended.

10 Repository permissions for the build identity

Where: *Project settings → Repositories → APIConnect-catalog → Security*

Find **<project> Build Service (<collection>)** — or *Project Collection Build Service* if *Project settings → Pipelines → Settings →* "*Limit job authorization scope to current project*" is off — and set:

Permission	Value	Why
Read	Allow	checkout
Contribute	Allow	push the `published/<stage>` tags
Create tag	Allow	same
Force push	Allow	the tags are moved with `git push --force refs/tags/…`; branches are protected by the policy anyway

In *Project settings → Pipelines → Settings* keep "Protect access to repositories in YAML pipelines" on; the pipeline only needs its own repository.

11 First run and Environments

Where: a PR that touches only `projects/example-project/dev/` (e.g. edit a description)

1. Open the PR. The *Build validation* check starts the pipeline; the first time it asks to **permit** access to the variable groups, secure files and agent pool — click *Permit* for each. Expect the four stages to pass and the publish stages to be skipped (PR builds never publish).
2. Complete the PR. The merge run validates again and then runs `publish_dev` and `publish_nightly` in parallel — both report "No tag published/… yet: bootstrapping" and publish the whole `dev/` folder. `publish_integ`, `publish_rc` and `publish_prod` also bootstrap their folders; if you do not want that yet, temporarily remove those folders from the example project first.
3. *Pipelines* → *Environments* now shows `apic-dev`, `apic-nightly`, `apic-integ`, `apic-rc`, `apic-prod`. On each: *Approvals and checks* → + → **Exclusive lock**. On `apic-prod` optionally add **Approvals** with the release managers.
4. In each API Manager confirm the products show *Published*; call `GET /example/health` through each gateway ingress with a subscribed application.

Checklist and troubleshooting

	Done when…
☐	`sudo -iu azagent bash -c 'command -v yamllint spectral yq oasdiff jq apic git'` prints seven paths
☐	The agent shows *Online* in pool `apic-linux`
☐	`apic catalogs:list --server <ingress.management> --org <org>` works from the agent VM for all three instances (after `apic iam-apikey`)
☐	`config/topology.yml` has no `CHANGE ME`; `AGENT_POOL` is set in `apic-ci.yml`
☐	Per instance: `credentials.json` downloaded and renamed; long-lived API key created and tested with `apic catalogs:list` (step 7)
☐	Login step in `templates/publish-stage.yml` matches the key type (`apic login --sso --apiKey` for toolkit keys)
☐	Variable groups `apic-shared`, `apic-dev`, `apic-test`, `apic-prod` exist; the API keys are padlocked
☐	Three secure files uploaded with the exact names `apic-toolkit-credentials-{dev,test,prod}.json`
☐	Branch policy on `main`: reviewers + build validation with the pipeline
☐	Build Service has Contribute, Create tag, Force push on the repository
☐	A PR turned green; the merge created `published/dev` and `published/nightly` tags (*Repos* → *Tags*)
☐	Exclusive lock on the five Environments

If something fails

Symptom	Fix
Pipeline never starts on a PR	Build validation policy missing or set to *Optional*/manual; the `pr:` YAML block does nothing on Azure Repos.
"No agent found in pool … that satisfies the demands"	`AGENT_POOL` misspelt, agent offline, or the pipeline is not authorized on the pool (pool → Security → Pipeline permissions).
"offline mode: 'X' is not installed on this agent"	Tool missing for the *agent user*: rerun `install-offline.sh`, check `/opt/azagent/.env` PATH, restart the service (`sudo ./svc.sh restart`).

`scripts/changeset.sh: Permission denied` (or any script)	Either the scripts lost their exec bit when the repo was imported from Windows (see the note in step 5 — the pipeline now runs `bash scripts/…`, so update to the current `apic-ci.yml` / `publish-stage.yml`), or the agent's work folder is owned by another user: you ran the agent once as `root` before installing the service as `azagent`. Fix: `sudo ./svc.sh stop && sudo chown -R azagent:azagent /opt/azagent && sudo ./svc.sh start`. In a custom container image, make sure the agent user can write `_work` and that `bash`, `git` and `realpath` (coreutils) are present.
Stage 3 “APIC validity” (or a publish step) hangs with no output; after *Cancel* the log suddenly shows everything validated	An `apic` call hit one of the toolkit's two first-run questions (license, live help) and waited on the agent's never-closed stdin; the cancel closed it, the question got EOF and the command finished. `--accept-license` alone leaves the live-help question pending and persists nothing, so a “first run” with only that flag does not help. Fix: `apic --accept-license --live-help=false version` once *as the agent user* (Dockerfile `RUN`, same `USER` / `HOME` the agent uses) and scripts at a version whose `scripts/lib.sh` passes both flags on every call with stdin closed and a timeout. Quick check on the agent: `timeout 90 apic --accept-license --live-help=false validate <product.yaml>` must finish from any HOME; `tests/e2e/run-tests.sh` section 1 prints the full picture.
“the apic toolkit is not on PATH”	Step 2 not done on this agent, or installed under another user's home. Put it in `/usr/local/bin`.
Variable group / secure file “not authorized”	Open the run → *Permit*, or Library → object → Pipeline permissions → add the pipeline.
Login: TLS / “self signed certificate in certificate chain”	Step 3: CA into the OS store and `NODE_EXTRA_CA_CERTS`; restart the agent service so it picks up `/etc/environment`.
Login: 401 / “user not found” / “invalid api key”	The key's identity is not a member of that instance's provider org, the key expired (`ttl`), or it was a single-use browser key. Recreate with step 7b and test interactively with the same `ingress.management`. Also check that the login *command* matches the key type (`iam-apikey` vs `login --sso --apiKey`).
`git push … refs/tags/published/…`: TF401027 / permission denied	Step 10 permissions for the right Build Service identity (project-scoped vs collection-scoped).
Stage *Skipped* although the folder changed	The change is not under `projects/<project>/<folder>/{apis,products}/`, or the tag already points at this commit (check *Repos* → *Tags*).
UniversalPackages / NodeTool task errors	They should not appear at all: confirm `APIC_TOOLKIT_SOURCE=preinstalled` and `TOOLS_MODE=offline`.

Day-2 reminders

Rotate the three API keys on a schedule (edit the variable, nothing in Git changes). After an API Connect fix-pack upgrade, reinstall the matching toolkit on the agent. To update the tools, rebuild the bundle on a connected machine and rerun `install-offline.sh`. Ingress changes are a one-line PR to `config/topology.yml`.